

Arquitectura - FINAL

Arquitectura general del sistema

La solución propuesta adopta una arquitectura orientada a servicios, inspirada en los principios de los microservicios, aunque sin implementar este paradigma en su forma completamente estricta. Esta decisión surge de la necesidad de la FAU de disponer de una plataforma modular, flexible y extensible, capaz de separar responsabilidades funcionales y facilitar la incorporación de diferentes tecnologías según los requerimientos de cada componente. A diferencia de una arquitectura monolítica, donde toda la lógica del sistema se encuentra concentrada en una única aplicación, la arquitectura seleccionada distribuye las responsabilidades en servicios independientes que interactúan mediante interfaces REST, colas de mensajería y mecanismos de coordinación basados en memoria compartida.

Es importante destacar que la arquitectura implementada no constituye una arquitectura de microservicios pura en términos de persistencia de datos, ya que varios servicios comparten una misma base de datos relacional. En particular, los servicios de chat, gestión de colecciones, procesamiento documental y notificaciones utilizan la base de datos, mientras que el servicio de autenticación dispone de una base independiente.

La distribución de responsabilidades entre servicios permite escalar de manera independiente aquellos componentes que presentan mayores exigencias computacionales. Por ejemplo, los servicios asociados a la inferencia de modelos de inteligencia artificial y al procesamiento de documentos pueden requerir recursos significativamente superiores a los necesarios para las funcionalidades administrativas o de gestión. Esta separación posibilita asignar recursos específicos —incluyendo aceleración mediante GPU cuando sea necesario— sin impactar el rendimiento ni la disponibilidad del resto de la plataforma.

Asimismo, la modularidad de la arquitectura favorece la evolución futura del sistema, permitiendo incorporar nuevas capacidades o integrar servicios adicionales sin necesidad de realizar modificaciones significativas sobre los componentes existentes. Este aspecto resulta especialmente relevante en el contexto institucional de la FAU, donde pueden surgir nuevos requerimientos vinculados al análisis de información, herramientas de apoyo a la toma de decisiones o integraciones con otros sistemas corporativos.

Finalmente, la arquitectura propuesta facilita la utilización de tecnologías heterogéneas, seleccionando para cada servicio aquellas herramientas que mejor se ajustan a sus necesidades específicas. En este proyecto, FastAPI se emplea para los componentes

relacionados con inteligencia artificial y procesamiento documental, aprovechando sus capacidades de programación asíncrona y su integración con el ecosistema de librerías especializadas en IA, como LangChain, LangGraph y Sentence Transformers. Por otro lado, Django se utiliza para la gestión de usuarios, permisos y lógica de negocio principal del sistema, beneficiándose de su ORM maduro, sus mecanismos integrados de autenticación y la robustez de su ecosistema para el desarrollo de aplicaciones empresariales. Esta combinación permite aprovechar las fortalezas de cada tecnología, manteniendo una separación clara entre los componentes de inteligencia artificial y los servicios centrales de la plataforma.

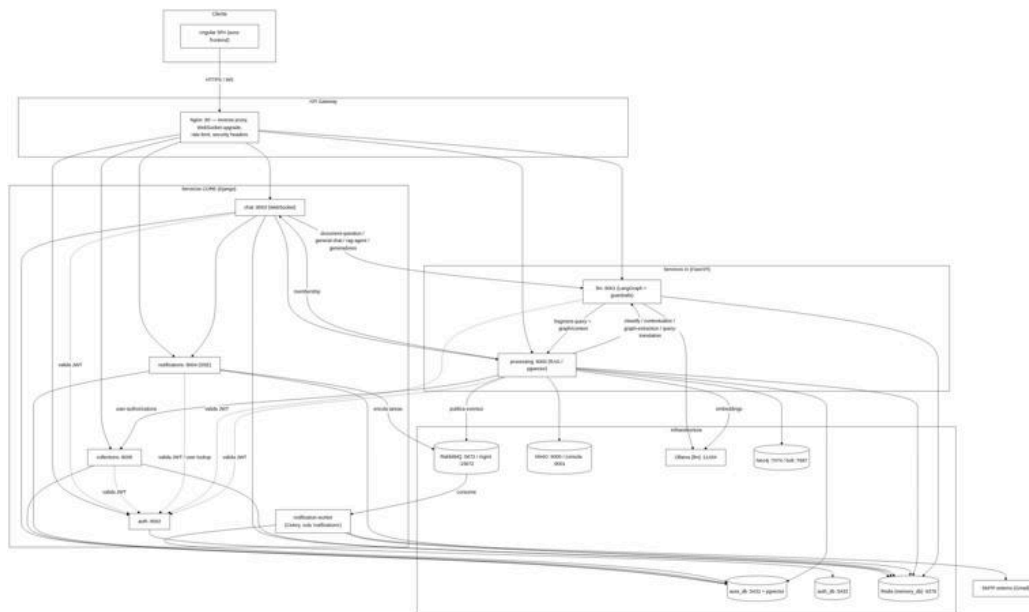
La decisión de mantener una base de datos independiente para el servicio de autenticación responde principalmente a criterios de seguridad, aislamiento y gestión de identidades. La información almacenada en la base de datos de autenticación contiene datos sensibles relacionados con usuarios, credenciales, roles y mecanismos de acceso, por lo que resulta conveniente separarla de la información operativa del resto de la plataforma. Este aislamiento reduce el impacto potencial de fallos o vulnerabilidades en otros servicios y permite evolucionar el componente de autenticación de forma independiente, facilitando futuras integraciones con proveedores externos de identidad o mecanismos avanzados de control de acceso.

Por el contrario, los servicios de chat, colecciones, procesamiento documental y notificaciones comparten una misma base de datos debido a la fuerte relación existente entre las entidades que gestionan. Dado que estos componentes forman parte del núcleo funcional de la plataforma y requieren acceder de manera frecuente a información común, una base de datos compartida simplifica la implementación, evita la duplicación de datos y reduce la complejidad asociada a la sincronización entre múltiples repositorios. Esta decisión representa un compromiso consciente entre los principios de desacoplamiento propios de una arquitectura de microservicios y la necesidad de mantener una solución más simple y manejable para el contexto y alcance del proyecto.

Diagrama de arquitectura y flujo del sistema

La solución desarrollada está compuesta por una capa de presentación, un punto central de acceso a los servicios, un conjunto de servicios backend especializados y diversos componentes de infraestructura que soportan el procesamiento, almacenamiento y consulta de la información. En términos generales, la arquitectura integra un frontend web, un API Gateway, seis servicios backend y múltiples tecnologías de soporte, incluyendo bases de datos relacionales, sistemas de mensajería, almacenamiento de objetos, bases de datos en memoria, motores de inteligencia artificial y una base de datos orientada a grafos.

El siguiente diagrama presenta la arquitectura de alto nivel del sistema, mostrando la interacción entre el frontend desarrollado en Angular, el API Gateway implementado con Nginx, los servicios backend desarrollados en Django y FastAPI, y los distintos componentes de infraestructura utilizados para el almacenamiento y procesamiento de datos.



La capa de presentación fue desarrollada utilizando Angular y constituye el principal punto de interacción entre los usuarios y la plataforma. Todas las solicitudes generadas desde esta interfaz son dirigidas inicialmente al API Gateway, encargado de centralizar y gestionar el acceso a los distintos servicios backend.

El API Gateway, implementado mediante Nginx, actúa como punto único de entrada al sistema. Su función principal consiste en enrutar las solicitudes hacia los servicios correspondientes, abstraer la complejidad de la infraestructura interna y proporcionar una interfaz unificada para el frontend. Adicionalmente, gestiona la propagación de conexiones WebSocket utilizadas por el módulo de chat en tiempo real, reenvía información relevante de la conexión mediante cabeceras de red y configura tiempos de espera extendidos para aquellos servicios cuyas operaciones pueden requerir varios minutos de procesamiento, particularmente los asociados a inteligencia artificial y procesamiento documental.

Los servicios especializados en inteligencia artificial y procesamiento documental fueron desarrollados utilizando FastAPI. Estos componentes son responsables de tareas como la extracción y transformación de contenido documental, la generación de embeddings vectoriales, la indexación y búsqueda semántica de información, la orquestación de agentes inteligentes y la generación de respuestas mediante modelos de lenguaje. Para cumplir estas funciones, interactúan con diversos componentes de infraestructura, incluyendo RabbitMQ para

la gestión de tareas asíncronas, PostgreSQL con la extensión pgvector para el almacenamiento de información estructurada y representaciones vectoriales, MinIO para la gestión de documentos y archivos, y Ollama como motor de inferencia para la ejecución de modelos de lenguaje de gran tamaño.

Por otra parte, los servicios CORE, desarrollados con Django, concentran la lógica de negocio principal de la plataforma. Entre sus responsabilidades se encuentran la gestión de usuarios, roles y permisos, la administración de chats y conversaciones, la gestión de colecciones documentales y el sistema de notificaciones. Estos servicios comparten la base de datos relacional principal del sistema (*aura_db*), permitiendo mantener la consistencia de la información y simplificando la gestión de entidades estrechamente relacionadas. La excepción corresponde al servicio de autenticación, que utiliza una base de datos independiente (*auth_db*) con el objetivo de aislar la información vinculada a identidades, credenciales y mecanismos de acceso.

Complementariamente, Redis es utilizado como infraestructura de soporte para operaciones que requieren baja latencia y acceso rápido a la información. Entre sus funciones se incluyen el almacenamiento temporal de datos en caché, la gestión de canales de comunicación para WebSockets y los mecanismos de publicación y suscripción empleados por el sistema de notificaciones en tiempo real.

Módulos tecnológicos y criterios de selección

La selección de las tecnologías utilizadas en el proyecto fue realizada por el equipo de desarrollo, dado que la Fuerza Aérea Uruguay no estableció restricciones ni preferencias tecnológicas específicas. En consecuencia, se definieron criterios orientados a maximizar la mantenibilidad, facilitar futuras tareas de operación y reducir la complejidad tecnológica de la solución.

Uno de los objetivos principales fue minimizar la cantidad de lenguajes y frameworks empleados, evitando una proliferación innecesaria de tecnologías que pudiera dificultar la evolución y el mantenimiento del sistema. Como resultado, la solución se concentra fundamentalmente en dos lenguajes de programación: Python para los servicios backend y TypeScript para la interfaz de usuario. De forma similar, la capa backend se construye sobre dos frameworks complementarios, Django y FastAPI, ambos pertenecientes al ecosistema Python.

Frontend

La interfaz de usuario fue desarrollada utilizando Angular, un framework de código abierto mantenido por Google que se caracteriza por su enfoque estructurado, modular y orientado al desarrollo de aplicaciones empresariales de gran escala.

Durante la etapa de diseño se evaluaron alternativas ampliamente utilizadas en la industria, como React y Vue. React ofrece una gran flexibilidad arquitectónica, pero requiere incorporar bibliotecas adicionales para funcionalidades esenciales como el enrutamiento, la gestión de estado o determinadas capacidades de la interfaz, lo que incrementa la complejidad de integración y mantenimiento. Vue, por su parte, presenta una curva de aprendizaje reducida y una menor complejidad inicial, aunque dispone de una menor adopción en entornos corporativos y de un ecosistema menos consolidado para aplicaciones de gran tamaño.

Considerando estos aspectos, Angular fue seleccionado por proporcionar una arquitectura consistente, un conjunto completo de herramientas integradas y una fuerte orientación hacia aplicaciones empresariales mantenibles a largo plazo.

Servicios de Inteligencia Artificial y Procesamiento Documental

Los componentes encargados del procesamiento documental y de las funcionalidades de inteligencia artificial fueron implementados utilizando FastAPI. Este framework fue seleccionado debido a su alto rendimiento, soporte nativo para programación asíncrona y excelente integración con el ecosistema de bibliotecas de inteligencia artificial disponible en Python.

Alternativas como Flask fueron descartadas debido a su naturaleza minimalista, que obliga a incorporar manualmente numerosas extensiones para funcionalidades habituales en aplicaciones complejas. También se evaluaron soluciones basadas en Node.js, como Express, aunque estas presentan una menor compatibilidad con las bibliotecas especializadas utilizadas para procesamiento documental, generación de embeddings y construcción de sistemas RAG.

FastAPI proporciona un equilibrio adecuado entre simplicidad de desarrollo, rendimiento y compatibilidad tecnológica, permitiendo integrar de forma eficiente herramientas como LangChain, LangGraph, Sentence Transformers y Docling, entre otras.

Servicios CORE

Los servicios responsables de la lógica de negocio principal de la plataforma fueron desarrollados utilizando Django. Este framework proporciona una infraestructura robusta para la construcción de aplicaciones empresariales, incorporando mecanismos de seguridad, autenticación, administración y persistencia de datos de forma nativa.

Durante el análisis tecnológico se consideraron alternativas como Spring Boot y Flask. Si bien Spring Boot ofrece excelentes capacidades para aplicaciones empresariales de gran escala, su adopción hubiera implicado introducir Java como tecnología adicional dentro del proyecto, aumentando la complejidad de desarrollo y mantenimiento. Flask, por otro lado, requeriría implementar manualmente numerosas funcionalidades que Django incorpora de manera integrada.

Por estos motivos, Django fue seleccionado como base para los servicios CORE, complementándose con Django REST Framework para la exposición de APIs REST y con Django Channels para la implementación de funcionalidades de comunicación en tiempo real mediante WebSockets.

Redis

Redis fue seleccionado como solución de almacenamiento en memoria debido a su elevado rendimiento y baja latencia. Dentro de la arquitectura cumple múltiples funciones: almacenamiento temporal en caché, soporte para la capa de comunicación de WebSockets, mecanismo de publicación y suscripción para notificaciones en tiempo real y gestión de bloqueos distribuidos utilizados durante la generación de respuestas mediante inteligencia artificial.

La centralización de estas funcionalidades en una única tecnología simplifica la arquitectura y reduce la necesidad de incorporar componentes adicionales.

API Gateway

Como punto único de entrada al sistema se seleccionó Nginx, una de las soluciones más utilizadas en arquitecturas distribuidas y orientadas a servicios. Su función consiste en recibir todas las solicitudes provenientes del frontend y redirigirlas al servicio correspondiente.

Además del enrutamiento, Nginx permite centralizar configuraciones relacionadas con seguridad, tiempos de espera, gestión de conexiones y soporte para protocolos como WebSocket. Su bajo consumo de recursos y su capacidad para manejar grandes volúmenes de conexiones concurrentes lo convierten en una alternativa adecuada para este tipo de arquitecturas.

Sistema de Mensajería

Para la gestión de procesos asíncronos se adoptó RabbitMQ como plataforma de mensajería. Su elección se fundamenta en la confiabilidad de entrega, soporte para reintentos y mecanismos avanzados de enrutamiento de mensajes.

Dentro del sistema, RabbitMQ es utilizado principalmente para coordinar las etapas del procesamiento documental y para gestionar el envío asíncrono de notificaciones por correo electrónico mediante Celery.

Se evaluaron alternativas como Redis y Apache Kafka. Sin embargo, Redis está orientado principalmente a escenarios de baja complejidad y Kafka resulta más apropiado para arquitecturas basadas en grandes flujos de eventos. RabbitMQ proporciona un equilibrio adecuado entre simplicidad operativa y confiabilidad para los requerimientos del proyecto.

Base de Datos Relacional

La persistencia principal de la plataforma se implementó utilizando PostgreSQL. Esta tecnología fue seleccionada por su estabilidad, madurez, capacidad de escalamiento y amplio soporte para extensiones avanzadas.

Además de almacenar la información transaccional del sistema, PostgreSQL permite integrar la extensión pgvector, utilizada para almacenar e indexar los embeddings generados durante el proceso RAG. Esta capacidad resultó determinante frente a otras alternativas relacionales, ya que posibilita combinar almacenamiento estructurado y búsqueda vectorial dentro de una misma plataforma tecnológica.

La solución emplea dos bases de datos diferenciadas: *aura_db*, utilizada por los servicios funcionales principales, y *auth_db*, dedicada exclusivamente a la gestión de identidades y autenticación.

Almacenamiento de Objetos

Para el almacenamiento de documentos se seleccionó MinIO, una solución que puede desplegarse íntegramente en infraestructura local.

La utilización de MinIO permite almacenar documentos originales de forma escalable y desacoplada de la base de datos relacional, manteniendo además el cumplimiento del requisito institucional de operar completamente dentro de la infraestructura de la organización, sin dependencia de servicios externos en la nube.

Motor de Inteligencia Artificial

El componente encargado de la ejecución de modelos de lenguaje es Ollama, una plataforma que permite desplegar y administrar modelos de inteligencia artificial de manera local.

Su elección estuvo motivada principalmente por los requisitos de seguridad y soberanía de los datos establecidos para la solución, que demandan un funcionamiento completamente

autónomo y sin dependencia de servicios externos.

La arquitectura implementada abstrae el acceso a los modelos mediante una fachada específica, permitiendo sustituir el modelo subyacente sin necesidad de modificar la lógica de negocio ni los agentes inteligentes. Esta decisión facilita futuras actualizaciones tecnológicas y reduce el acoplamiento entre la aplicación y el proveedor de inferencia.

Durante el desarrollo se evaluaron distintos modelos pertenecientes a las familias Gemma, Qwen, Llama, Mistral y DeepSeek. Los resultados obtenidos mostraron que la familia Gemma ofrecía el mejor equilibrio entre calidad de respuesta, desempeño en idioma español y consumo de recursos computacionales para el hardware objetivo. En consecuencia, se adoptó Gemma 4 para entornos productivos con aceleración mediante GPU y Gemma 3 de menor tamaño para entornos de desarrollo y pruebas ejecutados exclusivamente sobre CPU.

Versiones de las tecnologías principales

La Tabla 1 resume las versiones de los principales componentes tecnológicos utilizados en la implementación de la solución. Estas versiones corresponden al entorno de desarrollo empleado durante la realización del proyecto.

Componente	Versión
Python	3.13
Angular	20.1
Django	6.0.4
FastAPI	0.121
PostgreSQL	17
Redis	8.2
RabbitMQ	4.1
MinIO	RELEASE.2025-09-07T16-13-09Z
Ollama	Última versión estable
Neo4j	5 Community
Nginx	Alpine

Microservicios

La arquitectura de la solución está compuesta por seis servicios backend independientes, además del API Gateway que centraliza el acceso a la plataforma. Cada servicio posee responsabilidades bien definidas y se comunica con los demás mediante APIs REST y mecanismos de mensajería asíncrona cuando el procesamiento así lo requiere.

La gestión de usuarios, autenticación y autorización forma parte del servicio de autenticación, por lo que no existe un microservicio específico para usuarios. Asimismo, los mecanismos de monitoreo y observabilidad constituyen componentes transversales de la arquitectura y no un servicio independiente.

Servicio de procesamiento documental y recuperación semántica

Campo	Contenido
Nombre	aura-document-processing-service
Descripción	Servicio responsable de administrar el ciclo de vida de los documentos incorporados al sistema. Realiza el procesamiento del contenido, genera las representaciones vectoriales utilizadas por el sistema RAG y proporciona los mecanismos de recuperación semántica empleados durante las consultas.
Responsabilidades	Recepción y almacenamiento de documentos; extracción y normalización de contenido; segmentación del texto; generación e indexación de embeddings; recuperación de fragmentos relevantes mediante búsqueda semántica; coordinación del flujo de procesamiento documental y enriquecimiento opcional del grafo de conocimiento.
Tecnología	FastAPI
Base de datos	PostgreSQL (<i>aura_db</i> con soporte para vectores), MinIO para almacenamiento documental y RabbitMQ para procesamiento asíncrono.
Observaciones	Debido al elevado costo computacional del procesamiento documental, las tareas se ejecutan de

	forma asíncrona, permitiendo desacoplar el procesamiento de la interacción del usuario y mejorar la escalabilidad del sistema.
--	--

Servicio de LLM

Campo	Contenido
Nombre	aura-llm-service
Descripción	Servicio encargado de gestionar todas las interacciones con los modelos de lenguaje. Actúa como una capa de abstracción entre el resto del sistema y el motor de inferencia, centralizando la generación de respuestas y la ejecución de agentes inteligentes.
Responsabilidades	Recepción y procesamiento de consultas; orquestación de agentes especializados; integración con el motor de inferencia; generación de respuestas; ejecución de consultas RAG; generación de reportes, listas de verificación y resúmenes; soporte para respuestas en tiempo real mediante streaming.
Tecnología	FastAPI
Base de datos	No utiliza base de datos relacional. Emplea Redis para almacenar información temporal necesaria durante la ejecución de las consultas.
Observaciones	Este servicio desacopla la lógica de negocio del motor de inteligencia artificial, permitiendo reemplazar o actualizar el modelo de lenguaje sin modificar el resto de la arquitectura. La mayor parte de las solicitudes provienen del servicio de chat, aunque determinadas funcionalidades, como la generación de reportes y listas de verificación, son consumidas directamente por el frontend.

Servicio de autenticación

Campo	Contenido
Nombre	aura-auth-service
Descripción	Servicio encargado de la autenticación, autorización y administración de identidades dentro de la plataforma. Centraliza los mecanismos de control de acceso y gestiona la información de usuarios, roles y permisos, constituyendo el punto de entrada para la validación de credenciales.
Responsabilidades	Autenticación de usuarios; emisión, validación y renovación de tokens de acceso; gestión de usuarios, perfiles, roles y permisos; aplicación del modelo de control de acceso basado en roles (RBAC); registro de auditoría de eventos de autenticación y aplicación de políticas de seguridad, como el bloqueo temporal ante múltiples intentos fallidos de inicio de sesión.
Tecnología	Django
Base de datos	Base de datos PostgreSQL exclusiva (<i>auth_db</i>).
Observaciones	La utilización de una base de datos independiente permite aislar la información relacionada con identidades y credenciales del resto de los datos operativos de la plataforma, incrementando la seguridad y facilitando la evolución futura del mecanismo de autenticación, incluyendo una integración con servicios LDAP.

Servicio de chat

Campo	Contenido
Nombre	aura-chat-service
Descripción	Servicio responsable de gestionar la comunicación entre los usuarios y la interacción con el sistema de inteligencia artificial. Administra conversaciones persistentes, mensajes y funcionalidades

	colaborativas, proporcionando comunicación en tiempo real mediante WebSockets.
Responsabilidades	Creación y administración de conversaciones individuales y grupales; envío y recepción de mensajes en tiempo real; almacenamiento del historial de conversaciones; integración con el servicio de LLM para la resolución de consultas; transcripción de mensajes de voz; gestión de marcadores, mensajes fijados, comentarios, enlaces públicos para compartir conversaciones y exportación de contenido en distintos formatos.
Tecnología	Django
Base de datos	Base de datos PostgreSQL compartida (<i>aura_db</i>) y Redis como infraestructura para la comunicación en tiempo real.
Observaciones	El servicio implementa comunicación bidireccional mediante WebSockets, permitiendo que los usuarios reciban respuestas del sistema de forma inmediata. Asimismo, incorpora mecanismos de control de concurrencia y limitación de solicitudes para garantizar la estabilidad del servicio cuando múltiples usuarios interactúan simultáneamente con el sistema de inteligencia artificial.

Servicio de colecciones de documentos

Campo	Contenido
Nombre	<code>aura-document-collection-service</code>
Descripción	Servicio responsable de la organización lógica de los documentos almacenados en la plataforma. Permite agrupar la información en colecciones, administrar el acceso a las mismas y aplicar las políticas de clasificación utilizadas por la organización.

Responsabilidades	Creación y administración de colecciones documentales; asociación de documentos y usuarios a las distintas colecciones; gestión de niveles de clasificación y compartimentos de seguridad; control de acceso a la documentación según los permisos asignados a cada usuario.
Tecnología	Django
Base de datos	Base de datos PostgreSQL compartida (<i>aura_db</i>).
Observaciones	Este servicio constituye la base del modelo de seguridad documental implementado en la plataforma. La combinación de niveles de clasificación y compartimentos permite restringir el acceso a los documentos de acuerdo con las autorizaciones de cada usuario, siguiendo un esquema de control de acceso obligatorio (Mandatory Access Control, MAC).

Servicio de notificación

Campo	Contenido
Nombre	<code>aura-notification-service</code> + worker de Celery
Descripción	Servicio encargado de gestionar y distribuir las notificaciones generadas por los distintos componentes del sistema. Centraliza la comunicación de eventos relevantes hacia los usuarios, tanto dentro de la aplicación como mediante correo electrónico.
Responsabilidades	Recepción y procesamiento de eventos provenientes de los distintos servicios; envío de notificaciones en tiempo real; envío de correos electrónicos; administración de preferencias de notificación por usuario, tipo de evento y canal de comunicación.
Tecnología	Django

Base de datos	Base de datos PostgreSQL compartida (<i>aura_db</i>), Redis para la distribución de eventos en tiempo real y RabbitMQ para el procesamiento asíncrono de tareas.
Observaciones	Se trata de un servicio transversal consumido por el resto de los microservicios de la plataforma. La utilización de mecanismos asíncronos permite desacoplar la generación de eventos del envío efectivo de las notificaciones, mejorando el rendimiento general del sistema y evitando que operaciones de larga duración impacten sobre la experiencia del usuario.

Comunicación entre servicios

Los distintos microservicios que conforman la plataforma se comunican mediante tres mecanismos principales: servicios REST, colas de mensajería y una infraestructura de coordinación basada en Redis. La utilización de cada mecanismo depende de las características de la interacción, buscando un equilibrio entre simplicidad, rendimiento y desacoplamiento de los componentes.

La comunicación mediante servicios REST constituye el mecanismo principal para las operaciones síncronas que requieren una respuesta inmediata. A través de este mecanismo se realizan, entre otras operaciones, la validación de la autenticidad de los usuarios por parte del servicio de autenticación, la comunicación entre el servicio de chat y el servicio de inteligencia artificial para la generación de respuestas, y las consultas que el servicio de LLM realiza al servicio de procesamiento documental para recuperar el contexto necesario durante la ejecución del flujo RAG. Asimismo, los distintos servicios utilizan interfaces REST internas para intercambiar información y coordinar sus funcionalidades cuando la respuesta debe obtenerse de manera inmediata.

Para aquellas tareas cuyo procesamiento requiere un tiempo considerable o puede ejecutarse de forma desacoplada, la arquitectura utiliza RabbitMQ como sistema de mensajería. Este mecanismo se emplea principalmente durante el procesamiento documental, donde las distintas etapas del pipeline —como la extracción de contenido, la segmentación de documentos y la generación de embeddings— se ejecutan de manera asíncrona. De igual forma, el envío de notificaciones por correo electrónico se realiza mediante colas de mensajes, permitiendo que

estas tareas se procesen en segundo plano sin afectar el tiempo de respuesta percibido por el usuario.

Por otra parte, Redis proporciona una infraestructura de baja latencia para la coordinación de procesos en tiempo real y el almacenamiento temporal de información. Entre sus principales funciones se encuentran el soporte para las comunicaciones mediante WebSockets utilizadas por el servicio de chat, la distribución de notificaciones en tiempo real, la implementación de mecanismos de sincronización entre servicios —como los bloqueos que evitan la generación simultánea de múltiples respuestas para una misma conversación— y el almacenamiento en caché de información de acceso frecuente.

La combinación de estos tres mecanismos permite seleccionar el modelo de comunicación más apropiado para cada escenario. Mientras que las solicitudes REST garantizan respuestas inmediatas para las operaciones transaccionales, RabbitMQ desacopla las tareas de procesamiento intensivo y Redis facilita la coordinación de eventos en tiempo real. En conjunto, esta estrategia contribuye a mejorar la escalabilidad, el rendimiento y la mantenibilidad de la arquitectura propuesta.

Observabilidad

La arquitectura incorpora un conjunto de mecanismos de observabilidad destinados a supervisar el estado de los servicios, facilitar el diagnóstico de incidentes y evaluar el comportamiento general de la plataforma. A diferencia de los servicios funcionales del sistema, la observabilidad no se implementa como un microservicio independiente, sino como una infraestructura transversal que instrumenta todos los componentes de la solución. Esta infraestructura se despliega conjuntamente con la plataforma y opera íntegramente dentro del entorno local, sin depender de servicios externos.

El monitoreo operativo se basa en la recolección de métricas expuestas por cada uno de los microservicios. Estas métricas incluyen información sobre utilización de recursos, tiempos de respuesta, cantidad de solicitudes procesadas, errores y estado general de la aplicación. Prometheus realiza la recolección periódica de esta información y Grafana proporciona paneles de visualización que permiten analizar el comportamiento del sistema y verificar el cumplimiento de los objetivos de rendimiento definidos durante el desarrollo del proyecto.

En forma complementaria, todos los servicios generan registros estructurados que son centralizados mediante Filebeat y almacenados en Elasticsearch. Esta estrategia permite disponer de un repositorio unificado de eventos para facilitar las tareas de diagnóstico, análisis

de fallos y auditoría operativa. Asimismo, las solicitudes propagan identificadores de correlación entre los distintos microservicios, permitiendo reconstruir el recorrido completo de una petición a través de la arquitectura y simplificando la identificación de posibles puntos de falla.

El servicio de autenticación incorpora además un mecanismo de auditoría persistente para las operaciones relacionadas con la seguridad del sistema. Este registro almacena información sobre eventos relevantes, como autenticaciones, cambios de permisos y acciones administrativas, permitiendo mantener un historial de las operaciones realizadas y fortaleciendo los mecanismos de trazabilidad y control.

Desde el punto de vista de la disponibilidad, todos los servicios exponen puntos de verificación de estado (health checks), utilizados tanto por Docker Compose como por el API Gateway para comprobar periódicamente la disponibilidad de cada componente y detectar posibles fallos durante la operación.

Finalmente, debido a la incorporación de modelos de lenguaje y flujos RAG, la solución integra Arize Phoenix como plataforma de observabilidad especializada para aplicaciones de inteligencia artificial. Esta herramienta permite inspeccionar la ejecución de agentes, cadenas de procesamiento y consultas RAG, registrando información sobre los prompts enviados al modelo, el contexto recuperado, las respuestas generadas y los tiempos de ejecución de cada etapa. De esta forma, Phoenix complementa la observabilidad tradicional del sistema proporcionando visibilidad específica sobre el comportamiento de los componentes de inteligencia artificial, facilitando el análisis de calidad de las respuestas, la detección de errores y la optimización continua de los flujos de inferencia.

Grafo de conocimiento

Como complemento al mecanismo de recuperación semántica basado en embeddings, la arquitectura incorpora un grafo de conocimiento implementado sobre Neo4j. Su propósito es representar explícitamente las entidades identificadas en los documentos y las relaciones existentes entre ellas, permitiendo estructurar el conocimiento contenido en la documentación institucional.

Durante el procesamiento documental, el sistema puede extraer entidades relevantes —como personas, organizaciones, ubicaciones, unidades o conceptos específicos del dominio— junto con las relaciones que las vinculan. Esta información se almacena en un grafo, donde cada entidad corresponde a un nodo y cada relación representa un vínculo semántico entre dichos nodos.

La incorporación de un grafo de conocimiento complementa las capacidades del sistema RAG, ya que permite realizar consultas que no solo dependen de la similitud semántica entre textos, sino también de la estructura de las relaciones existentes en la información. De esta manera, es posible responder consultas orientadas al descubrimiento de conexiones entre entidades, identificar cadenas de relaciones y obtener una visión más estructurada del conocimiento almacenado.

Asimismo, el sistema contempla la traducción de consultas formuladas en lenguaje natural hacia consultas en Cypher, el lenguaje de consulta de Neo4j, permitiendo que los usuarios interactúen con el grafo sin necesidad de conocer su modelo interno de datos.

La integración de este componente amplía las capacidades analíticas de la plataforma al combinar dos enfoques complementarios para la recuperación de información: la búsqueda semántica mediante representaciones vectoriales y la exploración de relaciones mediante un grafo de conocimiento. Esta combinación proporciona una base más robusta para aplicaciones de inteligencia artificial que requieren tanto comprensión semántica como razonamiento sobre relaciones explícitas entre los distintos elementos del dominio documental.

Atributos de calidad

La arquitectura propuesta fue diseñada considerando un conjunto de atributos de calidad que responden a los requisitos funcionales y no funcionales definidos para el proyecto. La Tabla 2 resume cómo las principales decisiones de diseño contribuyen al cumplimiento de dichos atributos.

Atributo	Cómo lo aborda la arquitectura
Escalabilidad	La separación del sistema en servicios independientes permite escalar de manera selectiva aquellos componentes que presentan una mayor demanda de recursos, sin afectar el funcionamiento del resto de la plataforma. Asimismo, el procesamiento asíncrono reduce el impacto de las tareas de larga duración sobre la experiencia del usuario.
Disponibilidad	La arquitectura incorpora mecanismos de supervisión del estado de los servicios, recuperación automática ante fallos y un punto único de acceso que desacopla a los

	clientes de la organización interna del sistema, favoreciendo la continuidad del servicio.
Seguridad	Se implementan mecanismos de autenticación y autorización, control de acceso basado en roles y niveles de clasificación, protección de las comunicaciones entre servicios, auditoría de eventos relevantes y validación de la información recibida, contribuyendo a preservar la confidencialidad, integridad y trazabilidad de los datos.
Mantenibilidad	La separación de responsabilidades entre servicios, el uso de un conjunto reducido de tecnologías y la modularidad de la arquitectura facilitan la evolución del sistema, la incorporación de nuevas funcionalidades y el mantenimiento del código a largo plazo.
Portabilidad / operación local	La solución puede desplegarse íntegramente en infraestructura local mediante contenedores, reduciendo las dependencias externas y simplificando tanto la instalación como la migración entre distintos entornos de ejecución.
Observabilidad	La plataforma incorpora mecanismos de monitoreo, centralización de registros, auditoría y seguimiento de solicitudes, proporcionando información suficiente para detectar fallos, analizar el comportamiento del sistema y facilitar las tareas de mantenimiento y soporte.

16. Bibliografía

- Mozilla Developer Network. (2025, 24 de junio). *Introducción a Django*. MDN Web Docs. https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/Django/Introduction
- IBM. (s. f.). *¿Qué es Django?* IBM Topics. <https://www.ibm.com/es-es/topics/django>
- Amazon Web Services. (s. f.). *What is Django?* AWS. <https://aws.amazon.com/es/what-is/django/>
- Django Software Foundation. (2025). *Django documentation*. <https://docs.djangoproject.com/>
- Tiangolo. (s. f.). *FastAPI*. FastAPI. <https://fastapi.tiangolo.com/>

- Angular. (s. f.). *Overview – Angular*. angular.dev. <https://angular.dev/overview>
- IBM. (s. f.). *¿Qué son los microservicios?* IBM THINK. <https://www.ibm.com/mx-es/think/topics/microservices>
- Red Hat. (s. f.). *¿Qué son y para qué sirven los microservicios?* Red Hat. <https://www.redhat.com/es/topics/microservices>
- [Nginx.org](https://nginx.org). (s. f.). *Documentation*. NGINX. <https://nginx.org/en/docs/>
- RabbitMQ. (s. f.). *RabbitMQ*. <https://www.rabbitmq.com/>
- IBM. (s. f.). *What is Redis?* IBM THINK Topics. <https://www.ibm.com/think/topics/redis>
- PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. <https://www.postgresql.org/docs/current/>
- MinIO. (s. f.). *Enterprise AIStor Object Store*. MinIO Documentation. <https://docs.min.io/>
- Ollama. (s. f.). *Ollama*. <https://ollama.com/>
- LangChain. (s. f.). *LangChain / LangGraph documentation*. <https://python.langchain.com/>
- Neo4j. (s. f.). *Neo4j Graph Database*. <https://neo4j.com/>
- Google DeepMind. (s. f.). *Gemma*. DeepMind. <https://deepmind.google/models/gemma/>